L M U ☀ C M S I 3 8 0 2

LANGUAGES AND AUTOMATA II

HOMEWORK #1 Due: 2025-01-31

Learning Objectives

In this assignment you will demonstrate:

- ✅ Fluency in sketching **abstract syntax trees**
- ✅ The ability to craft a **grammar** from a natural language description of a programming language
- ✅ The ability to design and test a syntax with the **Ohm Editor**
- ✅ The creation of a **Node.js** project on GitHub, with a README, multiple imported packages, and 100% unit test coverage

Readings, Videos, and Self-Study

Although your team will be turning in only one submission, every team member is individually responsible for each of the following:

- ✅ Go through Nicky Case's *How to Remember Everything Forever-ish*.
- ✅ Read Atul Gawande's *The Checklist*.
- ✅ Make a pass over Gabe Robins' *A Brief History of Computing*.
- ✅ Work through the course notes on Computation, Language Design, and Ohm.
- ✅ What all embedded videos in the course notes above. Pay particular attention to the videos by Bret Victor and Alan Kay, as these will give you the right mindset for designing your own language in this course.
- ✅ Read the Introduction and Chapters 1–2 of [Bernhardt].
- ✅ Read the Ohm README, the Ohm Tutorial, and the four pages of documentation (Syntax, API, Philosophy, and Patterns & Pitfalls) linked from the main Ohm Documentation.

Submission Instructions

All homework in this class is to be done in your three to six-person group. **DO NOT "partition" the work.** Letting a team member do some exercises while you do others cheapens your educational experience and leaves you behind. It is best for each student to do all of the exercises, then meet as a group to (1) select the "best" answer for submission when more than one exist, and (2) teach each other. If one group member is unable to do one of the exercises, another group member can teach that student how it do it. Everyone is responsible for understanding the entire homework submission, so everyone should contribute to the entire assignment.

Select one student to upload *the* single submission for the entire team.

Your team's BrightSpace submission should be an attached PDF or text file with:

- ✅ The names of every student.
- ✅ For each student, an affidavit that each of the readings/watchings above were individually completed.
- ✅ Answers to each of the exercises below, numbered and neatly typeset. For problems involving code to write, host your code on Replit or OneCompiler (or similar) and provide the link to the hosting site in your BrightSpace submission.
- ✅ A link to the public GitHub repository hosting your team's project. This is to be a **public** GitHub repository named after the language you are creating. You will use this repo to host your compiler project, which will grow during each assignment. By the end of the term it will be amazing.

Add me as a committer to your repository. My github name is **rtoal**. This will help me help you during office hours if your project seems to be flailing.

Exercises

1. Give the AST for the following JavaScript program, using the level of detail we used during class. You can use my JS AST Viewer to guide you and check your work, but remember, the drawing you need to produce for full credit will be far less verbose than the tool's output. (Remember, the tool uses a third-party parser, esprima-next, that provides a complete ESTree-compliant AST, which is far more verbose than expected for hand-drawn ASTs.)

```
let [x, y] = [0, 0];
console.log(93.8 * 2 ** x + y);
```

2. Draw the AST for the following JavaScript program, using the level of detail we used during class.

```
import x from "x"
```

```
console.log(93.8 * {x} << x.r[z])
```

3. Draw the AST for the following JavaScript program, using the level of detail we used during class.

```
const x = x / {[x]: `${x}`}[x]("y")
```

4. For the following JavaScript fragment (not a complete program), draw the AST, using the level of detail we used during class.

```
class C {
  f({a, b: c}) {return ([a,f]) => C}
}
```

5. For each scenario below, give Ohm grammar rules for `Exp`, `Term`, `Factor`, and `Primary` that make the expression `-2**2`:
   a. Evaluate to 4.
   b. Evaluate to `-4`.
   c. Be a syntax error, while allowing `(-2)**2` and `-(2**2)` to be legal.

6. Astro is a really tiny language, so we'd like to make Astro++. This new language adds an if-statement, a while statement, a break statement, and relational operators. The while statement should start with the keyword `while`, followed by a test expression, followed by a block (a curly-brace delimited sequence of statements). The if-statement should start with the keyword `if`, followed by a test expression, then a block, then an optional else-part which is the keyword `else` followed by either a block or another if-statement. Neither the while statement nor the if statement should end with a semicolon. The break statement is only allowed to appear in a while statement's block. The relational operators are the same as those in Python and are to be NON-associative. All of the relational operators are on the same precedence level, lower than all other operators. Give the syntax of Astro++ using Ohm. Hint: Check your grammar with the Ohm Editor so that you don't needlessly throw away points.

For Your Project

In this class you will not only be writing a compiler—you will be writing a compiler for a language that you *design yourself*. You have wide latitude in designing your language: I want you to design a language you can actually love. The only requirements are: (1) it must be of sufficient complexity (i.e., enough interesting features, including features beyond the example languages we see in class), and (2) it requires significant

work in the static analysis phase. You can satisfy the latter requirement with a statically-typed language with static scoping (in addition to a few other checks of course), but it *may* be possible to design a dynamically typed language, provided you define a significant number of statically checked contextual constraints. If you go with a dynamically-typed language, you should check with me first to make sure your workload is fair.

- Complete the first pass of the design of your language. USE THE OHM EDITOR. Give your language description on the README.md file of a public GitHub repository you will be setting up for the project. You should have, in your README:

  - ✅ A nice logo (please put some care into this)
  - ✅ The name of your language, in a large font (unless the language name is readable from the image)
  - ✅ A one paragraph introduction (tell a story!)
  - ✅ A list of features
  - ✅ Lots of example programs. You can do the "your language on the left, JavaScript on the right" presentation style.

  A list of previous projects from students that have taken this class over the last decade or so can be found in the Course Notes on Language Design. You may find these helpful, but do beware that some of them may be lacking in some areas.

- Set up your project. Include a *package.json* which is nicely filled out with at least the proper project name and the names of all the team members in the `author` field. Add the property `"type": "module"` to allow for modern JavaScript. Ensure that `npm test` runs tests with coverage. For now, just fill the examples folder with a few example programs—at least five significant ones. You don't have to start writing the compiler, but since you want to make sure your project does *something*, make your main file simply print "Hello, world" or something silly or fun, but not the Bee Movie script or a Rickroll, as those are not yet old enough to be retro. Your test program needs at least one test, it can be as simple as testing that 1 = 1.

```
.
├── .gitignore
├── README.md
├── LICENSE
├── package.json
├── .prettierrc.json        -- optional, you don't have to have one
├── docs
|   └── logo.png
├── examples
|   └── ...                 -- at least 5 significant examples
├── src
|   └── (yourlangname).js   -- Just print hello, world
```

```
└── test
    └── compiler.test.js    -- Just assert 1===1 for now
```

Grading Rubric

To help you get a good score, here are all the things you will be graded on.

- Exercises (74 pts)
  - ✅ Exercise 1 (12 pts)
  - ✅ Exercise 2 (12 pts)
  - ✅ Exercise 3 (12 pts)
  - ✅ Exercise 4 (12 pts)
  - ✅ Exercise 5 (12 pts)
  - ✅ Exercise 6 (14 pts)
- Compiler Project (26 pts)
  - ✅ Project GitHub repo has same name as language (2 pts)
  - ✅ Project name appears in README (2 pts)
  - ✅ Logo appears in README (2 pts)
  - ✅ README has story of your language (2 pts)
  - ✅ README lists features of the language (2 pts)
  - ✅ README shows examples of programs (2 pts)
  - ✅ package.json contains all of the team members in the author property (2 pts)
  - ✅ package.json has "type": "module" (2 pts)
  - ✅ npm test will run, execute at least one test, and show 100% test coverage (2 pts)
  - ✅ Examples folder has at least 5 full example programs and covers the language features (2 pts)
  - ✅ Structure of project is exactly as specified on this page (2 pts)
  - ✅ .gitignore is present in repo and properly generated (let Github do it for you) (2 pts)
  - ✅ No extra stuff in repo (no node_modules, no .DS_Store, etc.) (2 pts)